

// Verilog Design

```
module register
#(
    parameter WIDTH = 8
)
(
    input logic      clk,
    input logic      rst,
    input logic      en,
    input logic [WIDTH-1:0] in,
    output logic [WIDTH-1:0] out
);

always_ff @(posedge clk or posedge rst) begin
    if (rst)
        out <= '0;
    else if (en)
        out <= in;
end

endmodule
```

// JSON Snippet for Associated Assertions

```
{
  "Assertion 1": {
    "clock signal condition": "@(posedge clk)",
    "disable condition": "disable iff (rst)",
    "logical expression": "en | => out == $past(in,1)",
    "Signals": ["en", "out", "in"],
    "Assertion Explanation": "when enable signal is set (1), output signal equals the last one clock cycle's input signal from the next clock cycle"
  },
  "Assertion 2": {
    "clock signal condition": "@(posedge clk)",
    "disable condition": "disable iff (rst)",
    "logical expression": "!en | => out == $past(out,1)",
    "Signals": ["en", "out"],
    "Assertion Explanation": "when enable signal is reset (0), output signal equals the last one clock cycle's output signal from the next clock cycle"
  }
}
```